

Temperatur übergeben – Anleitung für das RAG-Projekt

Wie die Temperatur des LLM gesetzt wird: in der .env, im Code oder einmalig im Terminal.

Verwandte Dateien:

modellauflösung.txt	- Workflow der Modellauflösung
provider_aufloesung.txt	- Workflow der Provider-Auflösung
modell_und_provider_aendern.txt	- Bedienungsanleitung Modell/Provider

Die Prioritätskette

```
ChatBot(temperature=...) > build_llm(temperature=...) > TEMPERATURE aus  
.env > 0.0
```

Wichtig: Die Temperatur wird NICHT über das Präfix oder CHAT_MODEL gesteuert. Sie hängt am Modell nur in einem Punkt – dem Sonderfall weiter unten.

Weg 1 – In der .env (gilt für jeden Start)

```
TEMPERATURE=0.7
```

Gelesen wird der Wert in config.py:

```
temperature: float = 0.0
```

Der Default ist 0.0, also deterministisch. Fehlt die Zeile, bleibt es dabei.

Der Validator daneben begrenzt den Bereich:

```
@field_validator("temperature")  
@classmethod  
def _check_temperature(cls, value: float) -> float:  
    if not 0.0 <= value <= 2.0:  
        raise ValueError("temperature muss zwischen 0.0 und 2.0 liegen")  
    return value
```

Ein Wert außerhalb von 0.0–2.0 lässt Settings mit einem ValidationError scheitern – nicht stillschweigend auf einen Default zurückfallen.

Das ist die Stelle für "ich will das Projekt grundsätzlich mit mehr Kreativität laufen lassen".

Weg 2 – Im Code (für einen einzelnen Bot)

Die Klasse ChatBot hat ein eigenes Feld dafür:

```
bot = ChatBot(temperature=0.7)
```

In `__post_init__` läuft das über `build_llm()` weiter. Bei `temperature=None` – dem Default der Klasse – wird der Parameter gar nicht erst übergeben und der Wert aus der `.env` greift.

Direkt am Client geht es auch:

```
llm = build_llm(temperature=0.7)
```

Weil `**overrides` in `build_llm()` als letztes zusammengeführt wird, schlägt dieser Wert alles aus `resolve_target()`:

```
kwargs.update(overrides)
```

Und einzeln in der REPL (im Python-Einzeiler):

```
uv run python -c "  
from rag_project.main_chatbot import ChatBot  
bot = ChatBot(temperature=0.9)  
print(bot.invoke('Nenne fuenf Namen fuer eine Python-Klasse.'))  
"
```

Weg 3 – Einmalig im Terminal (pro Prozess)

Es gibt KEIN `--temperature`-Flag und KEINEN `/temperature-Slash-Befehl`.
`main_chatbot.py` wertet nur `--model` und `--no-stream` aus.

Zwei Umwege, die trotzdem funktionieren:

VARIANTE A – Umgebungsvariable für diesen einen Prozess setzen

```
# Linux / macOS  
TEMPERATURE=0.7 uv run python -m rag_project.main_chatbot  
  
# Windows PowerShell  
$env:TEMPERATURE = "0.7"  
uv run python -m rag_project.main_chatbot
```

Das funktioniert, weil `load_dotenv(override=False)` gesetzt ist: Eine bereits in der Prozessumgebung stehende Variable gewinnt gegen die `.env`. Genau diese Eigenschaft macht diesen Weg überhaupt möglich.

VARIANTE B – nur prüfen, was am Ende herauskommt

```
uv run python -c "  
from rag_project.providers import resolve_target  
from rag_project.config import get_settings  
print('Aus .env  :', get_settings().temperature)  
print('Aufgeloest:', resolve_target('openai:gpt-5-pro').temperature)  
"
```

MERKE: In PowerShell muss der Wert in Anführungszeichen stehen (`$env:X = "0.7"`), sonst wird die Zahl als String übergeben und pydantic kann sie je nach Version nicht in float umwandeln.

MERKE: `$env:` gilt nur für das aktuelle Terminalfenster. Ein neues Fenster startet wieder mit dem Wert aus der `.env`.

Der Sonderfall: Modelle ohne temperature

Nicht jedes Modell akzeptiert temperature. Im Katalog steht dafür ein Feld:

```
supports_temperature: bool = True
```

Bei `openai:gpt-5-pro` ist es `False`. `resolve_target()` setzt die Temperatur dann auf `None`:

```
temperature = settings.temperature if target.supports_temperature else None
```

Und `build_llm()` lässt den Parameter in diesem Fall ganz weg:

```
if target.temperature is not None:
    kwargs["temperature"] = target.temperature
```

Prüfen ohne API-Aufruf:

```
uv run python -c "
from rag_project.providers import resolve_target
print('gpt-5-pro  :', resolve_target('openai:gpt-5-pro').temperature)
print('gpt-4o-mini:', resolve_target('openai:gpt-4o-mini').temperature)
"
```

Erwartet: `None` für `gpt-5-pro`, ein Zahlenwert für `gpt-4o-mini`.

VORBEHALT: Dieser Schutz greift nur beim regulären Weg. Ein direktes `build_llm(temperature=0.7)` landet ungeprüft im Client, weil `**overrides` am Ende zusammengeführt wird. Bei einem Modell mit `supports_temperature=False` wird der Wert dann trotzdem mitgeschickt – ob die API ihn ablehnt, zeigt sich erst beim Aufruf.

Praktische Werte

Fall	Temperatur	Warum
-----	-----	-----
RAG-Antworten aus Dokumenten	0.0	Zitat-Treue, keine Ausschmückung
Tests und Vergleiche	0.0	derselbe Input, derselbe Output
Chat, Erklärungen	0.3 – 0.7	natürlicheres Formulieren
Brainstorming	0.8 – 1.2	mehr Varianten

Für Antworten aus dem Handbuch mit Belegen ist 0.0 die richtige Wahl. Eine höhere Temperatur erhöht die Bereitschaft, Dinge zu formulieren, die nicht im Kontext stehen.

Kurzreferenz

Dauerhaft ändern	-> .env: TEMPERATURE=0.7
Pro Bot-Instanz	-> ChatBot(temperature=0.7)
Direkt am Client	-> build_llm(temperature=0.7)
Ein Prozess im Terminal	-> \$env:TEMPERATURE="0.7" (PowerShell) TEMPERATURE=0.7 ... (Linux/macOS)
Prüfen	-> uv run rag-project --show-config uv run python -c "from rag_project.config import get_settings; print(get_settings().temperature)"
Nicht verfügbar	-> --temperature (CLI-Flag existiert nicht) /temperature (Slash-Befehl existiert nicht)

Was noch fehlen könnte

Ein --temperature-Flag in main() und ein /temperature-Zweig in handle_command() wären je etwa fünf Zeilen. Beides ist derzeit nicht implementiert.